Swarm and Evolutionary Computation 108 (2026) 102514

Contents lists available at ScienceDirect

Swarm and Evolutionary Computation

journal homepage: www.elsevier.com/locate/swevo

Reinforcement learning enhanced zeroth-order optimization for large-scale multiobjective optimization problems☆

Yonglin He [a], Cheng He [a,*], Zhichao Lu [b], Ye Tian [c], Handing Wang [d], Yinglan Feng [e], Hongbin Li [a]

[a] State Key Laboratory of Advanced Electromagnetic Technology, School of Electrical and Electronic Engineering, Huazhong University of Science and Technology, Wuhan, 430074, China
[b] Department of Computer Science, City University of Hong Kong, Hong Kong, China
[c] School of Computer Science and Technology, Anhui University, Hefei, 230601, China
[d] School of Artificial Intelligence, Xidian University, Xi'an, 710071, China
[e] School of Artificial Intelligence, Shenzhen University, Shenzhen, 518061, China

ARTICLE INFO

Keywords:
Zeroth-order optimization
Large-scale optimization
Multiobjective optimization
Evolutionary computation
Reinforcement learning

ABSTRACT

"The curse of dimensionality" poses a major challenge to the scalability of multiobjective evolutionary algorithms (MOEAs) for large-scale multiobjective optimization problems (LSMOPs). Although existing MOEAs have achieved encouraging performance on problems with hundreds of decision variables, solving higher-dimensional LSMOPs remains computationally demanding. To mitigate these scalability challenges, we propose MOZO, an evolutionary zeroth-order (ZO) optimization method that combines zeroth-order directional estimation with a population-based MOEA backbone. Specifically, MOZO first estimates and aggregates objective-wise ZO gradients to provide search directions in high-dimensional decision spaces. It then refines these directions using acceleration and adaptation modules and embeds them within a population-based evolutionary search process to improve robustness against noisy estimates and objective conflicts. In addition, MOZO incorporates a reinforcement learning mechanism to adaptively configure suitable ZO and MOEA modules during the search process. Experimental results on benchmark LSMOPs with up to 100,000 decision variables demonstrate the effectiveness, efficiency, and scalability of MOZO compared with five state-of-the-art large-scale MOEAs. We further apply MOZO to the fine-tuning of `OPT-2.7B` and `OPT-6.7B` large language models on a single GPU, where the results show its practical potential under memory-constrained settings.

1. Introduction

Optimization aims to make systems as effective as possible and is a fundamental tool for driving progress in academia and industry [1]. Historically, efforts have primarily focused on single-objective optimization, in which the primary aim is to identify the most favorable solution with respect to a designated criterion or performance metric [2]. As system complexity increased, demand for multiobjective optimization increased to address trade-offs among conflicting goals. In particular, diverse real-world optimization challenges embody the essence of multiobjective optimization problems (MOPs), characterized by multiple conflicting objectives. In contrast to single-objective optimization, solving an MOP yields a set of Pareto-optimal trade-offs, providing comprehensive information to decision-makers.

Hundreds of multiobjective evolutionary algorithms (MOEAs) have been proposed, which follow the meta-heuristic principle for global optimization, and use population-based mechanisms to obtain a set of diverse optima [3]. Despite encouraging advances in addressing MOPs, the inherent nonlinearity and dimensionality of objective functions limit the efficiency and effectiveness of MOEAs [4]. The issue of nonlinearity arises from non-polynomial mapping functions, denoted as f. These create rugged landscapes with local optima, multimodal characteristics, biases, or concave features [5]. Fortunately, this challenge can be addressed by incorporating diversity-maintaining strategies, such as niche mechanisms or multi-population models. These approaches have improved the exploration capabilities of MOEAs. This comes with a modest increase in the number of function evaluations (FEs) required [6].

☆ This work was supported by the National Key Research and Development Program of China (No. 2022YFB2403800). The computation is completed in the HPC Platform of Huazhong University of Science and Technology.

* Corresponding author.
E-mail address: chenghe_seee@hust.edu.cn (C. He).

https://doi.org/10.1016/j.swevo.2026.102514
Received 1 April 2026; Received in revised form 21 June 2026; Accepted 17 August 2026
Available online 25 August 2026
2210-6502/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

The second challenge for MOEAs relates to "*the curse of dimensionality*" in optimization problems. This phenomenon occurs when the problem dimensionality increases linearly, leading to an exponential growth in the volume of the search space [7]. Such dimensionality growth leads to a sparsely populated search space, making it difficult for MOEAs to explore optima effectively [8]. Two specialized topics arise from the dimensionality issue: many-objective optimization problems (MaOPs) and large-scale MOPs (LSMOPs). MaOPs require handling more than three objectives ($m > 3$), which are difficult due to dominance resistance, and cause diminished selective pressure in conventional MOEAs [9,10]. In contrast, LSMOPs require optimization over 100 decision variables ($d \geq 100$), which pose greater challenges as $d \gg m$. Besides, the nonlinearity of objective functions in LSMOPs leads to complex variable interactions, resulting in rugged fitness landscapes and massive local optima [11].

Emerging applications, such as parameter optimization in power systems [12], the design and fine-tuning of deep learning models [13], and task alignment in large language models (LLMs) [14], need powerful MOEAs for LSMOPs. The goal is to manage both fine-grained detail and the expanded system scale in academia and industry. Significant attention has been paid to large-scale MOEAs for navigating the vast decision spaces and intricate variable interactions [15], which can be roughly categorized into three classes.

(1) Decision variable grouping based MOEAs: Approaches such as MOEA/DVA [16], LMEA [17], LSMOEA/D [18], and LERD [19], analyze the interactions among decision variables. By partitioning the high-dimensional optimization task into multiple low-dimensional subproblems $\Omega_k{}_{k=1}^{K}$, these methods effectively reduce the search space and enhance search efficiency through a divide-and-conquer approach. However, the decision variable analysis typically incurs a complexity of $\mathcal{O}(d^2)$, which can lead to excessive FEs and scalability challenges [20].

(2) Decision space reduction based MOEAs: Techniques such as WOF [21], LSMOF [22], and LMOEADS [23] construct dimensionality reduction-based mapping $\Phi : \Omega \rightarrow \tilde{\Omega}$ from the perspective of problem reduction. Nevertheless, this reduction can inadvertently trap the search within local or pseudominima [24], especially when critical regions of the original decision space are improperly excluded or distorted during the reduction process.

(3) Novel offspring generation based MOEAs: These methods enhance population search capability via adaptive reproduction operators, such as swarm intelligence in LMOCSO [25], distribution perturbation mechanism in DGEA [26], and subspace exploration in DSSEA [27]. Their primary goal is to enhance the exploration capability of MOEAs by generating promising offspring solutions, which require a large number of FEs to effectively converge [28].

Despite significant progress in addressing LSMOPs with hundreds, thousands, or even millions of decision variables [29], challenges persist in achieving effective and efficient search as the dimensionality of the decision space scales up. The use of gradient information is one of the most effective strategies for mitigating the complexities associated with large-scale decision variables. Gradient-based techniques such as Adam [30] and stochastic gradient descent (SGD) [31] are employed for training deep learning models, which are regarded as the most efficient methods for training large models. However, in black-box optimization scenarios, the gradient information is often inaccessible or extremely difficult to derive mathematically. Existing approaches, such as MOCGDE [32] and DE-based methods [33], often ignore the relationship between estimated gradients and evolutionary operators. Others rely on meta-heuristic strategies that introduce randomness without a clear mathematical foundation [34]. All these oversights limit their ability to navigate the intricacies of LSMOPs.

Recently, zeroth-order (ZO) optimization techniques have gained attention in deep learning [35]. ZO optimization bypasses the need for explicit gradients by approximating them via finite differences, resulting in algorithms that exhibit structural similarities to first-order (FO) methods [36]. These ZO methods boast theoretically unbiased gradient estimation and provable convergence properties on large language model (LLM) fine-tuning tasks [37]. However, ZO estimator errors increase quickly in high-dimensional spaces. Their scalability is also poor for many conflicting objectives, which limits convergence speed and applicability in large-scale settings [38].

Inspired by the potential of combining ZO optimization with MOEAs for LSMOPs, we explore this promising avenue. The integration is not straightforward, as it remains challenging to approximate the search directions of multiple conflicting objectives using ZO techniques, and to preserve the strengths of meta-heuristics in handling nonlinear and rugged landscapes while maintaining scalability in large-scale decision variables. To address these challenges, we propose an automated evolutionary ZO optimization method for large-scale multiobjective optimization. The main new contributions are summarized as follows:

- We propose MOZO, an evolutionary zeroth-order method for LSMOPs. Its mechanism-level novelty lies in coupling objective-wise ZO direction construction, multiobjective direction aggregation and conflict handling, and state-dependent module configuration within a population-based update process. Rather than claiming to eliminate the curse of dimensionality, MOZO aims to mitigate the resulting scalability difficulty by providing evaluation-efficient directional guidance in high-dimensional decision spaces.
- We develop a reinforcement learning-based configuration mechanism. It adaptively selects estimation, aggregation, acceleration, adaptation, and MOEA modules during the search process. This allows MOZO to adjust its search behavior online rather than relying on a fixed, manually selected configuration.
- Extensive experiments on benchmark LSMOPs demonstrate MOZO's effectiveness, efficiency, and scalability. Additional results on `OPT-2.7B` and `OPT-6.7B` fine-tuning tasks show MOZO's practical potential for memory-constrained LLM optimization scenarios.

The remainder of this paper is organized as follows. Section 2 reviews some concepts of optimization and outlines the motivation of this study. Details of the MOZO and its modules are elaborated in Section 3. Experimental study and application results are given in Section 4, and conclusions are drawn in Section 5.

2. Preliminaries and motivation

This section introduces the problem setting of large-scale multiobjective optimization, briefly reviews the background in zeroth-order optimization relevant to this work, and presents the motivation for MOZO.

2.1. Large-scale multiobjective optimization

We consider a minimization multiobjective optimization problem (MOP) of the form

$$\min_{\mathbf{x}\in\Omega} \mathbf{f}(\mathbf{x}) = \left(f_1(\mathbf{x}), f_2(\mathbf{x}), \ldots, f_m(\mathbf{x})\right)^{\top}, \qquad \mathbf{x} \in \mathbb{R}^d. \tag{1}$$

where $\Omega \subseteq \mathbb{R}^d$ is the feasible decision space, d is the number of decision variables, and $m \geq 2$ is the number of objectives.

Given two solutions $\mathbf{x}, \mathbf{y} \in \Omega$, $\mathbf{x}$ is said to Pareto-dominate $\mathbf{y}$, denoted by $\mathbf{x} \prec \mathbf{y}$, if $f_k(\mathbf{x}) \leq f_k(\mathbf{y})$ for all $k \in 1, \ldots, m$ and $f_j(\mathbf{x}) < f_j(\mathbf{y})$ for at least one objective j. A solution $\mathbf{x}^\star$ is Pareto optimal if there exists no $\mathbf{y} \in \Omega$ such that $\mathbf{y} \prec \mathbf{x}^\star$. The set of all Pareto-optimal solutions forms the Pareto set

$$\mathcal{P}^\star = \{\mathbf{x} \in \Omega : \nexists\, \mathbf{y} \in \Omega,\ \mathbf{y} \prec \mathbf{x}\}, \tag{2}$$

and its image in the objective space forms the Pareto front

$$\mathcal{F}^\star = \{\mathbf{f}(\mathbf{x}) : \mathbf{x} \in \mathcal{P}^\star\}. \tag{3}$$

A problem is typically regarded as a large-scale multiobjective optimization problem (LSMOP) when $d \geq 100$. In this regime, the main challenge arises not only from objective conflict but also from the rapidly expanding decision space. As d increases, candidate solutions become increasingly sparse, variable interactions become harder to exploit, and both convergence and diversity maintenance become substantially more difficult [28,39]. Therefore, search efficiency in the high-dimensional decision space is a central concern in the design of the LSMOP algorithm.

2.2. Zeroth-order optimization

When explicit gradients are unavailable, zeroth-order (ZO) optimization provides a natural alternative by constructing search directions from function evaluations alone. This makes ZO methods suitable for black-box optimization and attractive for large-model optimization due to their low memory overhead [35,36]. In particular, ZO optimization has recently received renewed attention in LLM fine-tuning, where methods such as MeZO show that ZO-SGD can achieve memory and computation costs comparable to inference [40,41].

A standard two-point zeroth-order gradient estimator is

$$\widehat{\nabla} f(\mathbf{x};\mathbf{u}) = \frac{\phi(d)}{\mu}\Big(f(\mathbf{x}+\mu\mathbf{u}) - f(\mathbf{x})\Big)\mathbf{u}, \tag{4}$$

where $\mathbf{u} \sim p$ is a random perturbation direction, $\mu > 0$ is the perturbation radius, and $\phi(d)$ is a dimension-dependent scaling factor. From a smoothing viewpoint, the corresponding objective can be written as

$$f_\mu(\mathbf{x}) = \mathbb{E}_{\mathbf{u}\sim p'}\left[f(\mathbf{x}+\mu\mathbf{u})\right], \tag{5}$$

where p' is the distribution induced by the perturbation scheme. Under standard conditions, (4) can be viewed as an unbiased gradient estimate of the smoothed objective f_μ.

Although ZO optimization provides useful directional information without explicit derivatives, its estimation quality often deteriorates in high-dimensional settings. In particular, the estimator variance typically increases with dimensionality, thereby reducing the reliability of the update direction [42]. Recent studies have attempted to alleviate this issue using more structured perturbations, such as low-rank variants [38]. Nevertheless, ZO optimization alone is not a complete large-scale multiobjective solver, as one must also handle noisy estimates, conflicting objectives, and changing search demands across optimization stages.

2.3. Reinforcement learning based configuration

The above observation suggests that a practical large-scale solver should not rely on a single zeroth-order strategy throughout the whole search process. Different estimation, aggregation, acceleration, and update rules may be preferred at different stages, depending on the current optimization state. This naturally motivates the use of reinforcement learning (RL) as a controller for state-dependent module configuration [43,44]. For instance, deep RL-assisted methods in operator selection have demonstrated superiority in constrained multiobjective optimization [45], adaptive knowledge transfer for evolutionary multitasking [46], and state-dependent optimization for scheduling [47].

We formulate dynamic module selection as a Markov decision process. At generation t, the optimizer is characterized by a state $\mathbf{s}_t$; the controller outputs an action $\mathbf{a}_t$ specifying the module configuration; and a reward r_t is returned based on the achieved improvement [48]. Among the available RL algorithms, proximal policy optimization (PPO) is adopted for its empirical stability and efficiency [49]. Let $\pi_\theta(\mathbf{a} \mid \mathbf{s})$ denote the actor policy and $V_\phi(\mathbf{s})$ the critic. PPO optimizes the clipped surrogate objective

$$\mathcal{L}_{\text{PPO}}(\theta) = \widehat{\mathbb{E}}_t\left[\min\left(\rho_t(\theta)\widehat{A}_t, \text{clip}\big(\rho_t(\theta), 1-\epsilon, 1+\epsilon\big)\widehat{A}_t\right)\right], \tag{6}$$

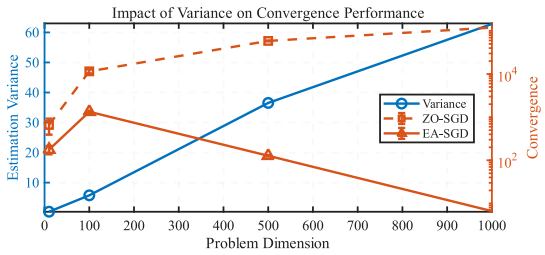


Fig. 1. The gradient estimation variance (left Y-axis) and the convergence performance (right Y-axis) obtained by ZO-SGD and EA-SGD on the CEC2008_F5 problem across different dimensions.

where

$$\rho_t(\theta) = \frac{\pi_\theta(\mathbf{a}_t \mid \mathbf{s}_t)}{\pi_{\theta_{\text{old}}}(\mathbf{a}_t \mid \mathbf{s}_t)}, \tag{7}$$

and the critic minimizes

$$\mathcal{L}_{\text{VALUE}}(\phi) = \widehat{\mathbb{E}}_t\left[\left(\widehat{R}_t - V_\phi(\mathbf{s}_t)\right)^2\right]. \tag{8}$$

In MOZO, RL is not used to replace optimization itself, but rather to adaptively configure the zeroth-order evolutionary search based on the current search state.

2.4. Motivation

Existing MOEAs for LSMOPs mainly rely on variable grouping [19], problem reformulation [22], or adaptive offspring generation [28]. Although effective, these paradigms still have important limitations: variable analysis can become expensive in high dimensions, reformulation can trap the search in pseudo-optima, and heuristic offspring generation may still require many function evaluations. This suggests that LSMOPs would benefit from more explicit directional guidance in the decision space.

ZO optimization provides such guidance without requiring explicit gradients, making it attractive for black-box large-scale optimization. However, its effectiveness is limited by estimation noise. In high-dimensional settings, the estimation error satisfies

$$\mathbb{E}\left[\left\|\widehat{\nabla} f(\mathbf{x}) - \nabla f(\mathbf{x})\right\|_2^2\right] = \mathcal{O}(d)\left\|\nabla f(\mathbf{x})\right\|_2^2 + \mathcal{O}\left(\frac{\mu^2 d^3 + \mu^2 d}{\phi(d)}\right), \tag{9}$$

which indicates that the quality of the zeroth-order direction may deteriorate substantially as the dimension increases [37]. As a result, directly applying single-point ZO updates may lead to unstable convergence in LSMOPs.

The comparison between ZO-SGD and EA-SGD in Fig. 1 indicates that introducing a population mechanism can substantially improve robustness under noisy zeroth-order updates. This motivates the central design of MOZO: instead of relying solely on zeroth-order (ZO) optimization, we embed ZO directional estimation into a population-based evolutionary search process, where ZO provides explicit search guidance, and the population mechanism absorbs estimation errors while preserving exploration and diversity. Moreover, since no single estimator, aggregation rule, acceleration mechanism, or update policy is likely to be uniformly effective across all problems and all search stages, MOZO further introduces reinforcement learning to adaptively configure the module combination online. Thus, MOZO is motivated as a dynamically configurable algorithm that combines zeroth-order directional guidance, evolutionary population robustness, and adaptive module selection for large-scale multiobjective optimization.

Fig. 1 shows two coupled phenomena. First, the variance of the ZO-SGD estimator increases with the number of decision variables, indicating that a single noisy direction becomes less reliable in high-dimensional spaces. Second, the convergence curve of EA-SGD remains

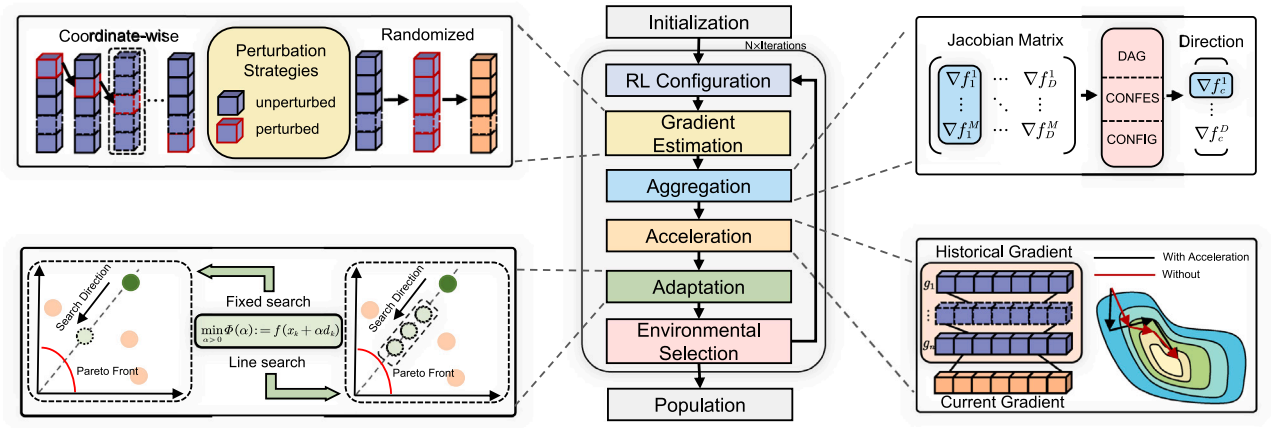


Fig. 2. The overall procedure of the proposed MOZO algorithm, which iteratively estimates gradients using coordinate-wise or randomized perturbation strategies, aggregates and accelerates search directions via the Jacobian matrix and historical gradients, and adapts the population through line or fixed search combined with environmental selection.

much more stable than that of standalone ZO-SGD under the same dimensional growth. This suggests that the evolution-based mechanism can absorb part of the estimation noise by retaining useful candidate solutions, thereby directly motivating the embedding of ZO directional estimation within population-based evolutionary search.

3. Method

We present MOZO, an adaptive evolutionary zeroth-order algorithm for large-scale multiobjective optimization. As illustrated in Fig. 2, MOZO augments a population-based search process with zeroth-order directional information and an online configuration controller. At generation t, the controller first specifies the configuration of the estimation, aggregation, acceleration, and adaptation modules. Given the selected configuration, MOZO computes an objective-wise zeroth-order Jacobian for each individual, transforms it into a unified search direction, refines it with historical information when necessary, and finally generates offspring through step-size-controlled updates. Environmental selection is then applied to form the next population, while the archive feedback is used to evaluate the configuration decision. This figure-to-method correspondence clarifies that the RL component does not replace evolutionary search; instead, it selects how the zeroth-order evolutionary search procedure is instantiated during different search stages.

Let $\mathcal{P}^t = \{\mathbf{x}_i^t\}_{i=1}^N$ denote the population at generation t, where each $\mathbf{x}_i^t \in \mathbb{R}^d$. The overall update of MOZO can be abstractly summarized as

$$\mathbf{y}_i^t = \mathrm{Adapt}\left(\mathbf{x}_i^t, \mathrm{Accel}\left(\mathrm{Agg}\left(\mathrm{Est}(\mathbf{x}_i^t)\right)\right)\right), \tag{10}$$

where Est, Agg, Accel, and Adapt denote the estimation, aggregation, acceleration, and adaptation modules, respectively. Table 1 summarizes all modules and their roles. This form highlights the role of MOZO as a direction-guided evolutionary optimizer: estimation constructs objective-wise zeroth-order signals, aggregation reconciles them into a search direction, acceleration improves its temporal consistency, and adaptation controls the update magnitude.

3.1. Zeroth-order estimation

The role of the estimation module is to construct an objective-wise zeroth-order Jacobian

$$\mathbf{J}(\mathbf{x}) = \begin{bmatrix} \widehat{\nabla} f_1(\mathbf{x})^\top \\ \vdots \\ \widehat{\nabla} f_m(\mathbf{x})^\top \end{bmatrix} \in \mathbb{R}^{m\times d}, \tag{11}$$

where $\widehat{\nabla} f_k(\mathbf{x})$ is a zeroth-order estimate of the gradient of the kth objective.

Table 1
Summary of MOZO modules and their roles.

Module	Candidate choices	Role in MOZO
Estimation	RGE, CGE, SignRGE, SignCGE, ComplexRGE, ComplexCGE	Constructs objective-wise zeroth-order directions and balances evaluation cost against directional fidelity.
Aggregation	DAG, CONFES, CONFIG	Converts the objective-wise Jacobian into a unified update direction while handling objective conflict at different levels.
Acceleration	FRCG, Adam, None	Uses historical directions or moments to reduce oscillation and improve temporal consistency.
Adaptation	Fixed search, line search	Controls the update magnitude and determines how aggressively offspring are generated.
Environmental selection	NSGA-II-style non-dominated sorting and crowding distance	Selects the next population from parent and offspring solutions while preserving convergence and diversity.
RL configuration	PPO controller	Selects the module configuration and continuous parameters according to the current optimization state.

MOZO considers two representative zeroth-order estimators. The first is the random gradient estimator (RGE), which perturbs the current point along a random direction $\mathbf{u} \sim p$ and computes

$$\widehat{\nabla}^{\mathrm{RGE}} f(\mathbf{x}) = \frac{\phi(d)}{\mu}\Big(f(\mathbf{x}+\mu\mathbf{u}) - f(\mathbf{x})\Big)\mathbf{u}, \tag{12}$$

where $\mu > 0$ is the perturbation radius and $\phi(d)$ is the dimension-dependent scaling factor. Since only one perturbation direction is required, RGE is computationally attractive in very high-dimensional settings.

The second is the coordinate-wise gradient estimator (CGE), defined as

$$\widehat{\nabla}^{\mathrm{CGE}} f(\mathbf{x}) = \frac{1}{\mu}\sum_{j=1}^{d}\Big(f(\mathbf{x}+\mu\mathbf{e}_j) - f(\mathbf{x})\Big)\mathbf{e}_j, \tag{13}$$

where $\mathbf{e}_j$ is the jth standard basis vector. Compared with RGE, CGE is usually more accurate because it estimates the contribution of each

coordinate separately, but it requires d perturbations and is therefore much more expensive.

To reduce communication and quantization overhead, MOZO also considers sign-compressed variants of the above estimators, namely SignRGE and SignCGE, obtained by applying the element-wise sign operator to (12) and (13), respectively. In addition, to improve numerical stability under small perturbation radii, MOZO considers complex-step variants, namely ComplexRGE and ComplexCGE, by evaluating the perturbed function along the imaginary axis and extracting the imaginary part.

In practice, zeroth-order estimates may vanish in flat regions or local traps, thus providing no effective search signal. To prevent stagnation, MOZO further introduces a DE-based fallback rule:

$$\widetilde{\nabla} f(\mathbf{x}) = \lambda(\mathbf{x})\,\widehat{\nabla} f(\mathbf{x}) + \big(1-\lambda(\mathbf{x})\big)(\mathbf{x}_{r_1} - \mathbf{x}_{r_2}), \tag{14}$$

where $\mathbf{x}_{r_1}, \mathbf{x}_{r_2} \in \mathcal{P}^t$ are randomly sampled individuals and $\lambda(\mathbf{x})$ is an indicator of whether the original estimate is nonzero. In this way, the estimation module provides a family of direction constructors with different trade-offs between evaluation cost, numerical stability, and directional fidelity.

3.2. Multiobjective aggregation

Given the objective-wise zeroth-order Jacobian $\mathbf{J}(\mathbf{x}) \in \mathbb{R}^{m\times d}$, the role of the aggregation module is to convert the m objective-wise directions into a single update vector $\mathbf{g}(\mathbf{x}) \in \mathbb{R}^d$. MOZO considers three representative aggregation strategies.

The first is direct aggregation (DAG), which forms a weighted linear combination of the objective-wise gradients:

$$\mathbf{g}^{\mathrm{DAG}}(\mathbf{x}) = \mathbf{w}^\top \mathbf{J}(\mathbf{x}), \qquad \mathbf{w} \in \Delta^{m-1}, \tag{15}$$

where $\mathbf{w} = [w_1, \dots, w_m]^\top$ is a nonnegative weight vector on the probability simplex $\Delta^{m-1} = \{\mathbf{w} \in \mathbb{R}^m_+ : \sum_{k=1}^m w_k = 1\}$. In this work, $\mathbf{w}$ is treated as a preference vector associated with the current individual or subproblem. This form is simple and efficient, but it may suffer from severe cancellation when the objective gradients are strongly conflicting.

The second is conflict-filtered aggregation (CONFES), which keeps gradient-based updates on conflict-free coordinates while reverting to an evolutionary difference on conflicting coordinates:

$$\mathbf{g}^{\mathrm{confes}}(\mathbf{x}) = \mathbf{D}(\mathbf{x}) \odot \big(\mathbf{w}^\top \mathbf{J}(\mathbf{x})\big) + \big(\mathbf{1} - \mathbf{D}(\mathbf{x})\big) \odot (\mathbf{x}_{r_1} - \mathbf{x}_{r_2}), \tag{16}$$

where $\mathbf{D}(\mathbf{x}) \in \{0,1\}^d$ is a binary conflict mask, $\odot$ denotes the Hadamard product, and $\mathbf{x}_{r_1}, \mathbf{x}_{r_2} \in \mathcal{P}^t$ are two randomly sampled individuals from the current population. For the jth coordinate, $D_j(\mathbf{x}) = 1$ indicates that the objective-wise directions on that coordinate are regarded as sufficiently consistent, in which case the aggregated gradient component is retained; otherwise, $D_j(\mathbf{x}) = 0$ and the update falls back to an evolutionary difference term. In this way, CONFES performs gradient-guided search only on coordinates with relatively low conflict.

The third is conflict-free inverse aggregation (CONFIG), which directly constructs a conflict-free direction from the full Jacobian geometry. Let $\mathbf{g}_k = \widehat{\nabla} f_k(\mathbf{x})$ denote the kth row direction of $\mathbf{J}(\mathbf{x})$, and define the normalization operator $\mathcal{U}(\mathbf{v}) = \mathbf{v}/(\|\mathbf{v}\| + \varepsilon)$. CONFIG first computes a normalized common direction

$$\mathbf{g}_u = \mathcal{U}\big([\mathcal{U}(\mathbf{g}_1), \dots, \mathcal{U}(\mathbf{g}_m)]^{-\top} \mathbf{1}_m\big), \tag{17}$$

where $(\cdot)^{-\top}$ denotes the pseudoinverse of the transposed matrix and $\mathbf{1}_m$ is the all-one vector. The final CONFIG update is then

$$\mathbf{g}^{\mathrm{CONFIG}}(\mathbf{x}) = \left(\sum_{k=1}^m \mathbf{g}_k^\top \mathbf{g}_u\right) \mathbf{g}_u. \tag{18}$$

Eqs. (17)–(18) provide a concise construction used in the main algorithm, while the supplementary material provides additional derivations and the bi–objective closed-form solution. Compared with DAG and CONFES, CONFIG handles objective conflict more explicitly by requiring the shared direction to be positively aligned with all objective-wise directions whenever such a direction exists.

The three strategies above represent three levels of conflict handling, i.e., direct combination, partial filtering, and explicit projection.

3.3. Direction acceleration

The aggregated direction $\mathbf{g}(\mathbf{x})$ already provides a usable search signal, but it may still be noisy or oscillatory due to zeroth-order approximation error. The acceleration module refines this direction by exploiting historical information. For notational simplicity, let $\mathbf{g}_t$ denote the aggregated direction of the current solution at iteration t, and let $\mathbf{d}_t$ denote the accelerated direction finally used for the update.

MOZO considers two representative acceleration strategies. The first is Fletcher–Reeves conjugate gradient (FRCG),

$$\mathbf{d}_t = -\mathbf{g}_t + \beta_t \mathbf{d}_{t-1}, \qquad \beta_t = \frac{\|\mathbf{g}_t\|_2^2}{\|\mathbf{g}_{t-1}\|_2^2}, \tag{19}$$

where $\mathbf{d}_{t-1}$ is the previous search direction and β_t is the Fletcher–Reeves coefficient. At the first iteration or whenever acceleration is restarted, we set $\mathbf{d}_t = -\mathbf{g}_t$. This strategy reduces zig-zag behavior by incorporating historical directional information.

The second is Adam-style acceleration, which maintains first- and second-order moment estimates of the aggregated direction sequence $\mathbf{g}_t$ and produces an adaptive direction as

$$\mathbf{d}_t = -\frac{\widehat{\mathbf{m}}_t}{\sqrt{\widehat{\mathbf{v}}_t} + \epsilon}, \tag{20}$$

where $\widehat{\mathbf{m}}_t$ and $\widehat{\mathbf{v}}_t$ are the bias-corrected first- and second-order moments, respectively, and $\epsilon > 0$ is a small constant for numerical stability. Here, we omit the standard recursive moment-update equations for brevity, since they follow the conventional Adam formulation.

3.4. Step-size adaptation

Once the search direction $\mathbf{d}_t$ is obtained, MOZO determines the update magnitude through the adaptation module. The resulting offspring is generated as

$$\mathbf{y}_i^t = \mathbf{x}_i^t + \alpha_t\, \mathbf{d}_i^t, \tag{21}$$

where $\alpha_t > 0$ is the step size.

MOZO considers two representative adaptation strategies. The first is fixed search, which uses a constant step size $\alpha_t = \alpha$. The second is line search, which selects α_t from a geometrically decayed candidate pool, e.g., $\alpha_t \in \{\gamma 2^{-s} : s \in S\}$ within a feasible interval. In this way, the adaptation module controls how strongly the refined direction should influence the current solution. Together with the acceleration module, it determines not only where the search moves but also how aggressively it moves.

3.5. RL-based configuration

The modules above provide multiple candidate operators, each with distinct strengths and weaknesses. Instead of fixing a single combination throughout the optimization process, MOZO introduces an RL controller that adaptively configures the module sequence.

At generation t, the controller observes the current optimization state $\mathbf{s}_t$, outputs an action $\mathbf{a}_t$ specifying the module configuration, and receives a reward r_t proportional to the improvement in archive quality. Concretely, $\mathbf{s}_t \in \mathbb{R}^8$ encodes both convergence and diversity information: normalized hypervolume, spacing-based diversity, population–archive distance, archive size ratio, population spread, objective improvement rate, remaining evaluation progress, and normalized problem dimensionality. The action $\mathbf{a}_t \in \mathbb{R}^6$ is decoded into the estimation

Table 2
Compact implementation details and parameter settings of MOZO.

Item	Setting
Population/archive size	$N = 50$ for benchmark experiments and $N = 10$ for LLM experiments; the external archive is truncated to N by crowding distance.
Evaluation budget	$1000d$ for module and scalability analysis; $100d$ for comparisons with representative large-scale MOEAs.
Function-evaluation accounting	One FE denotes one objective-vector evaluation $\mathbf{f}(\mathbf{x})$. The stored objective values of current population members are reused; RGE, SignRGE, and ComplexRGE require one additional perturbation evaluation per individual, whereas CGE, SignCGE, and ComplexCGE require d additional perturbation evaluations per individual. All trial points generated by fixed-point or line search are counted, and every algorithm terminates under the same FE budget, measured by the actual number of objective-vector calls.
MOEA backbone	Parent–offspring merge followed by non-dominated sorting and crowding-distance truncation.
Action decoding	Six-dimensional actions select the four module types and control the perturbation radius μ and the base step size.
Acceleration restart	Restart at the first generation, when the acceleration module changes, or when the generated update is invalid.
PPO settings	8-dimensional state; actor–critic MLP with three hidden layers; discount 0.95, entropy weight 0.02, horizon 256, mini-batch 32, four epochs, and clipping factor 0.3.
Reward	Archive hypervolume increment between consecutive generations.
Reproducibility package	The implementation code, PPO training settings, random seeds, static module configurations, and LLM fine-tuning scripts are publicly available; see the Code Availability section.

Algorithm 1: Overall Procedure of MOZO

Input: Objective vector $\mathbf{f}$, population size N, budget B, policy π_θ, and module pool $\mathcal{M}$.
Output: Archive $\mathcal{A}$.

Initialize $\mathcal{P}$, evaluate $\mathbf{f}(\mathcal{P})$, set $\mathcal{A} \leftarrow \mathrm{ND}(\mathcal{P})$, initialize $\mathcal{H}$, and set $FE \leftarrow N$
while $FE < B$ **do**
  $\mathbf{s} \leftarrow \mathrm{EncodeState}(\mathcal{P}, \mathcal{A}, FE/B)$
  $\mathbf{a} \leftarrow \pi_\theta(\mathbf{s})$
  $(\mathrm{Est}, \mathrm{Agg}, \mathrm{Accel}, \mathrm{Adapt}, \mu, \eta) \leftarrow \mathrm{Decode}(\mathbf{a}; \mathcal{M})$
  $\mathcal{Q} \leftarrow \emptyset$
  foreach $\mathbf{x}_i \in \mathcal{P}$ **do**
    $\mathbf{J}_i \leftarrow \mathrm{Est}(\mathbf{f}, \mathbf{x}_i, \mu)$
    if $\mathrm{Invalid}(\mathbf{J}_i)$ **then**
      $\mathbf{J}_i \leftarrow \mathrm{DEFallback}(\mathcal{P})$
    end
    $\mathbf{g}_i \leftarrow \mathrm{Agg}(\mathbf{J}_i, \mathcal{P})$
    $(\mathbf{d}_i, \mathcal{H}_i) \leftarrow \mathrm{Accel}(\mathbf{g}_i, \mathcal{H}_i)$
    if $\mathrm{Restart}(\mathbf{d}_i)$ **then**
      $\mathbf{d}_i \leftarrow \mathbf{g}_i$
      reset $\mathcal{H}_i$
    end
    $\mathbf{y}_i \leftarrow \mathrm{Adapt}(\mathbf{x}_i, \mathbf{d}_i, \eta)$
    evaluate $\mathbf{f}(\mathbf{y}_i)$ and update FE
    $\mathcal{Q} \leftarrow \mathcal{Q} \cup \{\mathbf{y}_i\}$
  end
  $\mathcal{R} \leftarrow \mathcal{P} \cup \mathcal{Q}$
  $\mathcal{A}^+ \leftarrow \mathrm{ArchiveUpdate}(\mathcal{A} \cup \mathcal{R}, N)$
  $\mathcal{P} \leftarrow \mathrm{EnvironmentalSelection}(\mathcal{R}, N)$
  if *training mode* **then**
    update π_θ using $HV(\mathcal{A}^+) - HV(\mathcal{A})$
  end
  $\mathcal{A} \leftarrow \mathcal{A}^+$
end
return $\mathcal{A}$

method, aggregation method, acceleration strategy, adaptation strategy, perturbation radius μ, and base update step. The first four action dimensions are discretized into the candidate operators in Table 1, while the last two dimensions control the update scale and search granularity.

The reward is defined as the archive hypervolume gain,

$$r_t = HV(\mathcal{A}_t) - HV(\mathcal{A}_{t-1}), \tag{22}$$

where $\mathcal{A}_t$ denotes the archive at generation t. HV is computed with a fixed reference point. The controller is trained with PPO under a standard actor–critic architecture, where the actor parametrizes the policy $\pi_\theta(\mathbf{a} \mid \mathbf{s})$ and the critic estimates the value function $V_\phi(\mathbf{s})$. During training, each episode corresponds to one MOZO run under a fixed evaluation budget. The collected state–action-reward trajectories are used to update the actor and critic using (6) and (8).

During the testing, the trained policy is loaded and performs only forward inference at each generation, so the control overhead is small compared with objective evaluations and zeroth-order estimation. From this perspective, RL is not used to replace the optimizer itself, but to determine how the zeroth-order evolutionary search procedure should be instantiated at different stages of the search. This makes MOZO a dynamically configurable optimizer. To improve the reproducibility, Table 2 summarizes the fixed implementation settings that are not fully specified by Algorithm 1.

4. Empirical experiments

We systematically test the performance of MOZO and its static variants on GLSMOP test suites, and analyze the contributions of each component and the effectiveness of the RL-based dynamic selection mechanism. Then, the scalability of MOZO with respect to problem dimensionality is examined, followed by comprehensive comparisons with five state-of-the-art large-scale MOEAs. Finally, the performance of MOZO relative to baseline methods in multiobjective fine-tuning of LLMs under GPU memory constraints is evaluated to assess its practical applicability.

4.1. Parameter settings

To assess the overall performance of MOZO, we use two representative test suites, i.e., GLSMOP [50] and TREE [12]. The GLSMOP test suite is selected for its variable-interaction irregularities that reflect the complex features of LSMOPs, whereas the TREE problems capture real-world properties of LSMOPs. In general, we select nine GLSMOP problems with 100, 1000, 10,000, and 100,000 decision variables, and TREE problems with 12,000 and 30,000 decision variables, each configured with two or three objectives for comprehensive testing. The maximum number of FEs is set to $1000\times d$ (for the module combination and decision space scalability analysis) and $100\times d$ (for performance comparisons with representative MOEAs). Five state-of-the-art (SOTA) large-scale MOEAs, including problem reformulation based LMOEADS [23], gradient based MOCGDE [32], novel offspring generation based LMOCSO [25], efficient sampling based FLEA [29], and decision variable analysis based LERD [19], are selected as baselines for comparison. The inverted generational distance (IGD) [51] and hypervolume (HV) [52] are adopted as the performance metrics. Each algorithm is executed 20 times independently for each test instance. The population size is set to 50 for storage considerations. The Wilcoxon rank-sum test with a significance level of 0.05 is conducted to assess the statistical significance of performance differences [53].

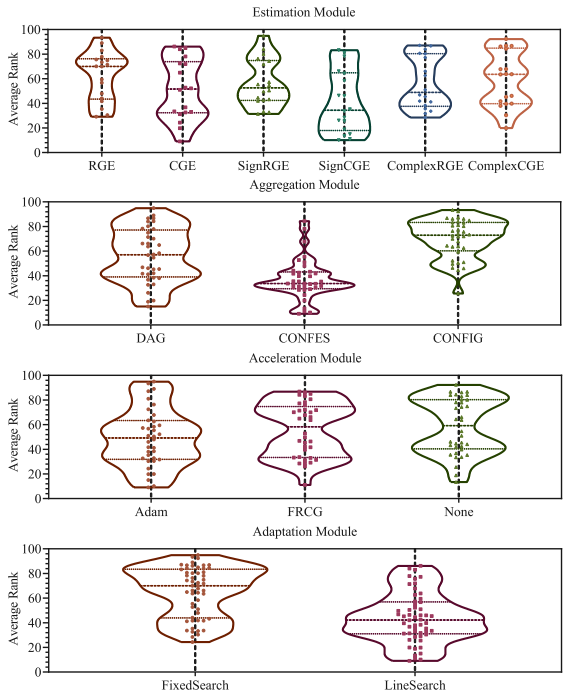


Fig. 3. Violin plots of average ranks achieved by MOZO variants with different module configurations on bi-objective GLSMOP problems with 1000 decision variables. For each test instance, all configurations are ranked according to IGD, and their ranks are averaged over the nine problems. Lower average ranks indicate better overall performance.

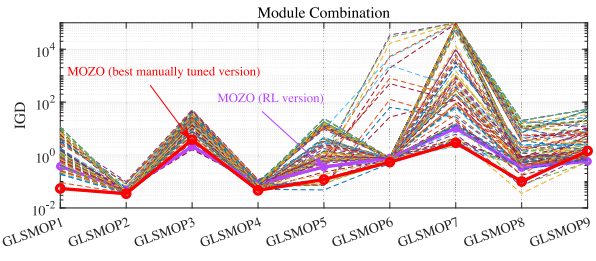


Fig. 4. Module combination results of MOZO on nine GLSMOP problems with 1000 decision variables. The results in purple bold correspond to the MOZO variant employing RL-based dynamic module configuration, whereas those in red bold correspond to the best manually tuned configuration.

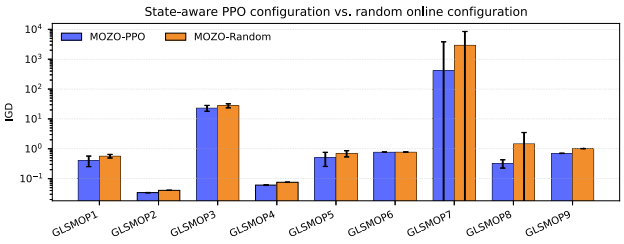


Fig. 5. Validation of PPO-guided dynamic configuration against random online module selection on bi-objective GLSMOP problems with 1000 decision variables. MOZO-Random uses the same candidate modules and action encoding as MOZO, but randomly samples the online configuration at each generation.

4.2. Combinatorial analysis

To investigate the role of each module in MOZO, we analyze each module, compare static module combinations with dynamic RL-driven selection, and assess the scalability of the MOZO algorithm.

Module Importance: The importance of each module in MOZO is tested. The average ranks achieved by MOZO variants with different configurations are given in Fig. 3. For each test instance, all configurations are ranked according to IGD, and the ranks are then averaged across the nine bi-objective GLSMOP problems with 1000 decision variables. Therefore, a lower average rank indicates better overall performance. A module is considered more influential when its candidate choices lead to clearly shifted or widely spread rank distributions. As can be observed, the estimation and aggregation modules show more pronounced differences among their candidates than the acceleration and adaptation modules, indicating that how MOZO obtains and reconciles objective-wise directions has the strongest effect on final performance.

Static vs. Dynamic Configuration: To assess the efficacy of dynamic module selection, we constructed 108 static MOZO variants and compared them with the RL-driven adaptive counterpart. Fig. 4 reports the IGD values obtained by all static configurations on nine GLSMOP problems, where each colored line denotes a distinct manually tuned variant.[1] The red bold curve corresponds to the best manually tuned configuration, which nearly dominates the other static variants across the benchmark set. This observation suggests that some static configurations remain competitive across multiple GLSMOP instances. In comparison, the RL-based MOZO (purple bold) achieves the best or near-best performance across the tested configurations, despite the absence of any manual prior. Such results indicate that RL-guided dynamic selection enables MOZO to adapt its module combination to varying problem landscapes, achieving robust and competitive performance without relying on handcrafted priors.

To examine whether the improvement is due to the learned state-dependent controller rather than online module switching alone, we introduce a MOZO-Random baseline. MOZO-Random uses the same action space and candidate modules as MOZO, but samples the six continuous action variables uniformly at random in each generation instead of using the trained PPO policy. Fig. 5 shows a comparison on the nine bi-objective GLSMOP problems with 1000 decision variables. Under this setting, PPO-guided MOZO achieves lower IGD than MOZO-Random on all nine problems, indicating that the state information exploited by PPO leads to more effective configuration decisions than random switching under the same module pool and action encoding.

Scalability: We examine the scalability of MOZO in handling LSMOPs with scaled-up decision variables. Here, MOZO is tested on bi-objective GLSMOP problems with 100, 1000, 10,000, and 100,000 decision variables. The number of FEs is adjusted proportionally to the problem dimension to ensure a fair comparison. Fig. 6 presents the IGD values obtained on test instances with varying numbers of decision variables. MOZO maintains stable IGD performance as the dimensionality increases, thereby demonstrating its scalability and applicability to extremely large-scale scenarios.

4.3. General performance

To assess the effectiveness and efficiency of MOZO for solving LSMOPs with varying numbers of decision variables, we compare MOZO against five SOTA large-scale MOEAs.

The achieved IGD results on relatively low-dimensional GLSMOP instances, i.e., $d = 100$ and $d = 1000$, are summarized in Table 3. The performance differences among the six algorithms are relatively small, and the best mean IGD results are more evenly distributed across

[1] The detailed IGD results of the 108 static MOZO variants are provided in the supplementary materials.

Table 3
Statistics of IGD results achieved by MOZO and five compared algorithms on Bi-objective GLSMOP problems with 100 and 1000 decision variables.

Problem	D	LMOEADS	MOCGDE	LMOCSO	FLEA	LERD	MOZO
GLSMOP1	100	1.3801e+0(1.29e−1)−	3.7146e+0(1.13e+0)−	6.3193e−1(1.09e−1)−	1.7744e+0(3.31e−1)−	1.1075e+0(1.71e−1)−	2.2898e−1(1.19e−1)
	1000	1.4233e+0(1.62e−1)−	1.0243e+1(1.19e+0)−	1.6156e+0(2.27e−1)−	2.1578e+0(1.17e−1)−	2.3070e+0(1.62e−1)−	4.1119e−1(1.58e−1)
GLSMOP2	100	1.6311e−1(3.43e−3)+	2.7579e−1(4.72e−3)−	1.8020e−1(5.59e−3)+	2.0581e−1(9.34e−3)−	1.7637e−1(4.85e−3)+	1.9814e−1(9.09e−3)
	1000	2.4596e−2(4.09e−4)+	4.0620e−2(1.37e−4)−	2.8673e−2(7.17e−4)+	3.4250e−2(1.87e−3)≈	2.9279e−2(7.54e−4)+	3.4038e−2(2.60e−4)
GLSMOP3	100	1.0062e+3(1.25e+3)−	2.0823e+1(7.36e+0)−	3.1814e+0(1.86e+0)+	2.6010e+1(2.75e+0)−	2.6419e+1(3.58e+0)−	1.5431e+1(5.55e+0)
	1000	2.7607e+2(5.14e+2)−	3.1249e+1(4.87e+0)−	2.7839e+0(1.08e+0)+	3.0248e+1(1.16e+0)−	2.9257e+1(1.36e+0)−	2.3134e+1(5.13e+0)
GLSMOP4	100	3.0967e−1(8.16e−3)+	4.0349e−1(1.06e−2)−	2.9029e−1(2.78e−2)+	3.2711e−1(1.34e−2)+	2.6340e−1(2.62e−2)+	3.7393e−1(1.04e−2)
	1000	5.3105e−2(3.80e−4)+	7.8142e−2(4.07e−4)−	5.4114e−2(6.56e−4)+	6.0248e−2(2.08e−3)≈	5.5938e−2(7.66e−4)+	6.0827e−2(1.58e−3)
GLSMOP5	100	4.2253e+0(5.24e−1)−	3.3746e+0(1.35e+0)−	1.3557e+0(3.82e−1)−	4.0990e+0(4.88e−1)−	2.5442e+0(7.83e−1)−	7.4213e−1(3.65e−1)
	1000	2.9687e+0(1.64e−1)−	1.7494e+1(4.60e+0)−	3.5930e+0(4.89e−1)−	4.4404e+0(4.63e−1)−	4.4537e+0(5.12e−1)−	5.0681e−1(2.51e−1)
GLSMOP6	100	5.7650e+3(6.38e+3)−	1.1537e+0(2.45e−2)−	9.0941e−1(4.18e−2)+	1.0536e+0(5.40e−2)−	1.0747e+0(5.09e−2)−	9.8302e−1(2.30e−2)
	1000	1.4175e+4(1.15e+4)−	7.4647e−1(7.49e−2)+	7.6842e−1(3.23e−3)≈	7.6090e−1(5.76e−3)≈	7.6982e−1(4.87e−3)≈	7.7197e−1(1.08e−3)
GLSMOP7	100	1.8424e+3(9.29e+2)−	3.0867e+4(3.35e+4)−	8.6319e+1(1.01e+2)+	1.8564e+3(6.18e+2)−	1.5550e+1(1.28e+1)+	1.0602e+3(1.50e+3)
	1000	8.3535e+2(4.99e+1)−	1.7126e+4(1.65e+4)−	2.6703e+3(1.52e+3)−	2.0785e+3(4.11e+2)−	1.2935e+1(4.25e+0)+	4.1470e+2(3.41e+3)
GLSMOP8	100	3.0934e+0(4.90e−1)−	2.0558e+0(1.01e+0)−	1.1744e+0(2.98e−1)−	3.2938e+0(6.00e−1)−	4.9828e−1(2.60e−2)+	5.5057e−1(2.55e−1)
	1000	2.1209e+0(5.27e−2)−	1.1268e+1(5.61e+0)−	2.5798e+0(3.28e−1)−	3.3693e+0(3.63e−1)−	5.0987e−1(1.18e−2)−	3.2713e−1(1.02e−1)
GLSMOP9	100	4.6655e+0(2.31e+0)−	1.8840e+0(7.80e−1)+	1.4279e+0(4.82e−1)+	8.0718e+0(2.84e+0)−	3.3545e+0(1.04e+0)≈	2.0909e+0(1.26e−1)
	1000	1.0776e+1(1.09e+0)−	3.8542e+0(5.07e+0)−	4.3210e−1(1.70e−2)+	9.3425e+0(1.95e+0)−	6.5815e+0(1.61e+0)−	7.0186e−1(4.93e−3)
+/ − / ≈		4/14/0	2/16/0	11/7/1	0/15/3	7/9/2	

+, −, and ≈ indicate that the result of the compared algorithm is significantly better than, worse than, or statistically similar to that of MOZO under the Wilcoxon rank-sum test at the 0.05 significance level, respectively. The minimum mean result in each row is highlighted in gray.

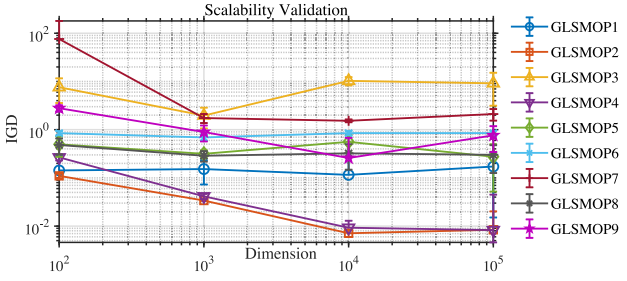


Fig. 6. IGD values achieved by MOZO on bi-objective GLSMOP problems with 100 to 100,000 decision variables. The x-axis denotes problem dimensionality, and the y-axis reports IGD, where lower values indicate better convergence and diversity.

different problems. Specifically, on the 18 relatively lower-dimensional problems, MOZO achieves significantly better results than LMOEADS, MOCGDE, LMOCSO, FLEA, and LERD in 14, 16, 7, 15, and 9 cases, respectively. Among these, LMOCSO and LERD maintain competitive performance in the relatively lower-dimensional decision space.

The achieved IGD results on scaled-up GLSMOP instances, i.e., $d = 10,000$ and $d = 100,000$, are summarized in Table 4. As shown, the performance gap widens rapidly as the problem dimensionality increases to 10,000 and 100,000. In the scaled-up scenarios, MOZO achieves significantly better IGD results than LMOEADS, MOCGDE, LMOCSO, FLEA, and LERD in 16, 18, 15, 15, and 16 out of 18 problems, respectively.[2] The main benchmark tables focus on representative bi-objective GLSMOP settings, where MOZO shows clear scalability advantages. The supplementary tri-objective GLSMOP results show a more nuanced trend. Based on the pairwise +/ − / ≈ summaries over 36 tri-objective GLSMOP instances and five compared algorithms, MOZO obtains significantly better results in 109 out of 180 HV comparisons and 125 out of 180 IGD comparisons, while being significantly worse in 10 and 36 comparisons, respectively. Most of these less favorable cases are observed against LMOCSO, which outperforms MOZO in 7 HV and 19 IGD tri-objective comparisons. These results indicate that MOZO is generally competitive, particularly in large-scale bi-objective scenarios, while its advantage can be less pronounced on some tri-objective instances.

To further validate the generalization capability of MOZO on problems with complicated landscapes, we evaluate all algorithms on bi-objective TREE problems with $d = 12,000$ and $d = 30,000$ decision variables. The HV results are summarized in Table 5. As shown, MOZO demonstrates superior performance across most TREE instances, achieving significantly better HV values than the compared algorithms in 6 out of 10 test cases. Notably, LMOEADS shows competitive performance on TREE3 and TREE4, while other SOTAs generally struggle with the structural characteristics of TREE problems. These results confirm MOZO's effectiveness in handling diverse landscapes.

The convergence profiles of all compared algorithms on GLSMOP problems with 10,000 decision variables are presented in Fig. 7. It can be observed that MOZO generally achieves faster and more stable convergence rates than others across most problems. MOEAs that rely on problem decomposition (e.g., LMOEADS) often stagnate or even exhibit an increasing IGD due to pseudominima traps.

We further visualize the non-dominated solutions from median IGD runs on GLSMOP1, GLSMOP5, and GLSMOP9 with 10,000 decision variables in Fig. 8. Consistent with the convergence profiles, LMOEADS struggles to generate well-distributed solutions along the PF, often failing to effectively approach the true front. LERD, while competitive on smaller-scale problems, is severely limited by the cost of variable analysis and fails to complete runs on some large-scale instances. The other algorithms also show slow progress and limited convergence in large-scale landscapes. In contrast, MOZO generally generates solution sets that are well-converged and well-distributed, thereby approximating the true PF more closely in most displayed cases. These results collectively support the scalability and convergence capability of MOZO, especially for LSMOPs, where traditional MOEAs often fail to maintain sufficient search efficiency and diversity.

To validate the efficiency of MOZO, the runtime consumption on GLSMOP instances with 10,000 and 100,000 decision variables is displayed in Fig. 9.[3] MOZO consistently ranks among the most efficient algorithms across both settings. In particular, gradient-based algorithms such as MOCGDE and MOZO generally exhibit lower runtime overheads. In contrast, LERD incurs the highest computational cost, largely due to its variable analysis and grouping operations, which become

[2] The IGD and HV results achieved in tri-objective GLSMOP and TREE test suites are shown in the supplementary materials.

[3] The runtime consumption on GLSMOP with 100 and 1000 decision variables is given in the supplementary materials.

Table 4
Statistics of IGD results achieved by MOZO and five compared algorithms on Bi-objective GLSMOP problems with 10,000 and 100,000 decision variables.

Problem	D	LMOEADS	MOCGDE	LMOCSO	FLEA	LERD	MOZO
GLSMOP1	10,000	1.5121e+0(1.13e−1)−	1.0132e+1(3.29e+0)−	2.2506e+0(2.53e−1)−	2.2590e+0(9.86e−2)−	2.5075e+0(3.40e−1)−	2.8134e−1(8.87e−2)
	100,000	1.6737e+0(2.29e−1)−	1.0766e+1(1.94e+0)−	2.2861e+0(2.45e−1)−	2.2512e+0(1.26e−1)−	NaN	2.8620e−1(1.33e−1)
GLSMOP2	10,000	9.0796e−3(2.79e−4)−	1.0012e−2(5.23e−5)−	8.6162e−3(8.18e−5)−	9.9051e−3(3.34e−4)−	8.6169e−3(6.57e−5)−	7.6284e−3(9.90e−5)
	100,000	8.1667e−3(3.71e−4)−	8.1472e−3(9.30e−5)−	7.3727e−3(1.32e−5)−	9.4336e−3(6.47e−4)−	NaN	7.1581e−3(1.04e−4)
GLSMOP3	10,000	2.8371e+1(1.58e−1)+	4.3272e+1(1.73e+1)−	2.4225e+0(5.55e−1)+	2.9127e+1(8.73e−1)+	2.9567e+1(1.42e+0)+	3.2368e+1(3.00e+0)
	100,000	2.8680e+1(3.20e−1)≈	8.8222e+2(1.54e+3)−	2.1334e+0(1.38e−1)+	2.8700e+1(2.57e−1)≈	NaN	2.8790e+1(3.84e+0)
GLSMOP4	10,000	1.2531e−2(5.23e−4)−	1.5572e−2(5.44e−5)−	1.1604e−2(9.39e−5)−	1.3386e−2(2.36e−4)−	1.2032e−2(4.82e−5)−	9.6725e−3(3.53e−4)
	100,000	9.4296e−3(3.35e−4)−	1.0267e−2(6.99e−5)−	8.3958e−3(2.29e−5)≈	1.0484e−2(5.53e−4)−	NaN	8.3742e−3(2.12e−4)
GLSMOP5	10,000	2.8215e+0(6.48e−2)−	2.0164e+1(7.44e+0)−	4.4177e+0(4.68e−1)−	4.2006e+0(3.94e−1)−	4.0466e+0(4.45e−1)−	7.0708e−1(1.24e−1)
	100,000	2.7939e+0(3.66e−2)−	2.4580e+1(6.34e+0)−	5.1860e+0(2.97e−1)−	4.1037e+0(2.82e−1)−	NaN	6.3590e−1(1.91e−1)
GLSMOP6	10,000	2.0328e+4(1.80e+4)−	7.4481e−1(3.27e−6)−	7.4471e−1(7.43e−5)−	6.8606e−1(1.23e−1)≈	7.4447e−1(2.12e−4)−	6.8014e−1(6.05e−4)
	100,000	1.9732e+4(1.69e+4)−	7.4828e−1(5.17e−6)−	7.3328e−1(7.21e−5)−	7.2878e−1(6.31e−1)−	NaN	6.7014e−1(5.97e−4)
GLSMOP7	10,000	6.2284e+2(1.40e+1)−	5.4712e+3(9.82e+3)−	6.4968e+3(4.26e+3)−	2.3375e+3(4.45e+2)−	1.4638e+1(5.23e+0)+	2.3144e+2(2.15e+2)
	100,000	6.2810e+2(1.19e+1)−	2.1287e+3(8.16e+3)−	1.0138e+4(1.01e+4)−	3.0189e+3(8.21e+2)−	NaN	1.9262e+2(1.55e+2)
GLSMOP8	10,000	2.1134e+0(2.30e−2)−	9.6857e+0(7.98e+0)−	3.3552e+0(4.01e−1)−	2.8935e+0(4.96e−1)−	5.4224e−1(6.17e−2)−	3.1120e−1(1.78e−1)
	100,000	2.3731e+0(6.21e−2)−	1.1687e+1(8.90e+0)−	4.2580e+0(5.31e−1)−	2.9910e+0(5.02e−1)−	NaN	3.2374e−1(2.01e−1)
GLSMOP9	10,000	1.4111e+1(2.14e+0)−	5.0017e+1(2.55e+1)−	9.0061e−1(2.09e−1)−	7.9287e+0(3.91e+0)−	6.3448e+0(1.30e+0)−	2.8684e−1(1.37e−1)
	100,000	2.2721e+1(2.21e+0)−	4.3968e+1(3.67e+1)−	7.1137e−1(1.82e−1)−	8.3967e+0(4.58e+0)−	NaN	2.6637e−1(1.18e−1)
+/ − / ≈		1/16/1	0/18/0	2/15/1	1/15/2	2/16/0	

+, −, and ≈ indicate that the result of the compared algorithm is significantly better than, worse than, or statistically similar to that of MOZO under the Wilcoxon rank-sum test at the 0.05 significance level, respectively. The minimum mean result in each row is highlighted in gray. NaN indicates that the algorithm failed to terminate in three days.

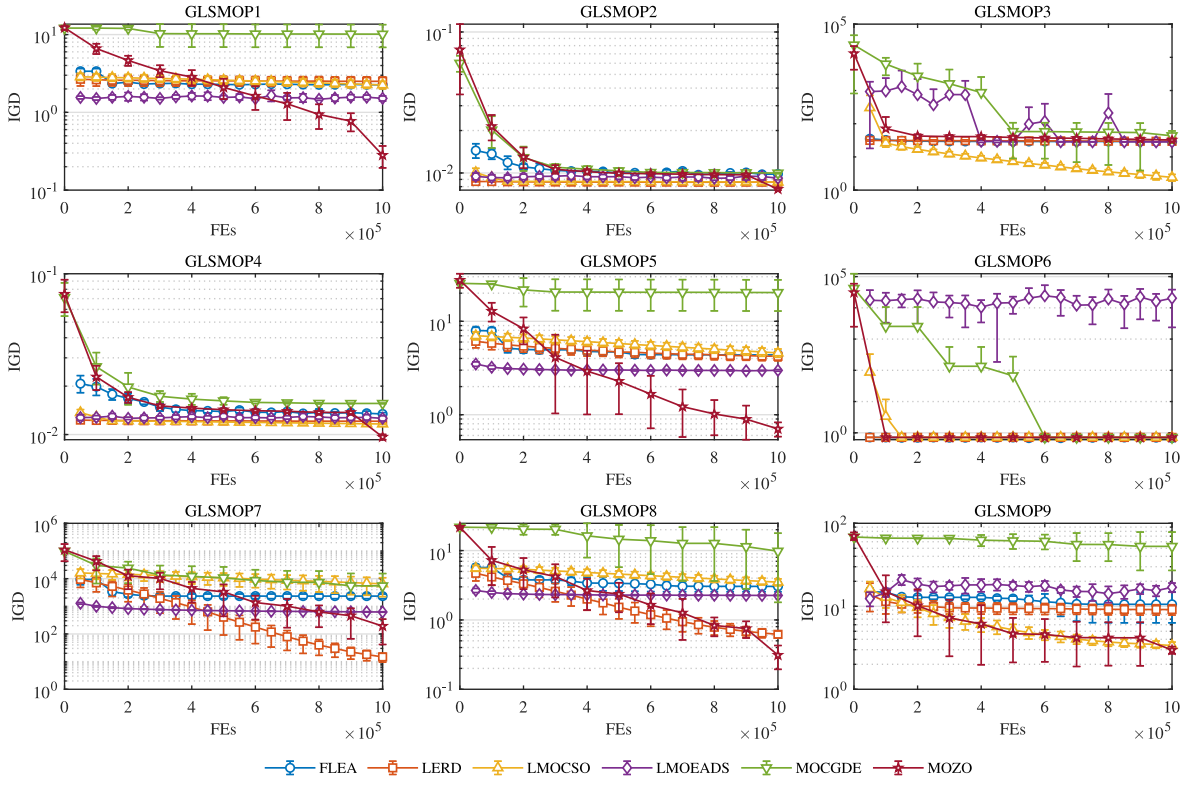


Fig. 7. Convergence profiles of MOZO and five compared algorithms on bi-objective GLSMOP problems with 10,000 decision variables, where color bars denote the standard deviations.

prohibitively expensive as dimensionality increases. These results further underscore the practical scalability and efficiency of MOZO in large-scale scenarios.

To complement the runtime curves, we further provide an empirical runtime profiling table in the supplementary material. The profiling separates state/action control, zeroth-order estimation and aggregation, offspring update with objective evaluation, and archive/environmental maintenance on a representative GLSMOP instance. The measured shares are 3.58%, 59.06%, 33.05%, and 2.04%, respectively, with the remaining 2.27% spent on algorithm-level overhead. These results indicate that the runtime is dominated by zeroth-order estimation and offspring/objective evaluation, whereas PPO-based online control is not the main computational bottleneck.

4.4. LLM fine-tuning

To examine the scalability of MOZO in real-world applications, we compare MOZO and LMOCSO against two baselines (the ZO optimizer, named ZO-SGD, and the FO optimizer, named FO-SGD) on LLM fine-tuning tasks. For MOZO and LMOCSO, the population size is set to 10 as only one GPU is available. All experiments adopt full-parameter fine-tuning using the `OPT-2.7B` and `OPT-6.7B` LLMs [54] on the NVIDIA V100 GPU.

The performance evaluation is conducted on five standard natural language understanding (NLU) tasks that cover a diverse range of reasoning and comprehension skills. Specifically, SST-2 tests the model's ability to detect nuanced sentiment in movie reviews through

Table 5
Statistics of HV results achieved by MOZO and five compared algorithms on Bi-objective TREE problems with 12,000 and 30,000 decision variables.

Problem	D	LMOEADS	MOCGDE	LMOCSO	FLEA	LERD	MOZO
TREE1	12,000	8.3224e−1(1.30e−4)−	7.3759e−1(5.13e−3)−	8.4252e−1(2.03e−4)−	8.3268e−1(2.98e−4)−	7.8766e−1(3.35e−3)−	8.4755e−1(2.36e−4)
	30,000	8.4372e−1(6.55e−5)−	7.3611e−1(3.91e−3)−	8.5147e−1(1.98e−4)−	8.4405e−1(1.59e−4)−	7.7728e−1(1.59e−4)−	8.5794e−1(2.03e−4)
TREE2	12,000	8.4955e−1(1.01e−4)−	7.9894e−1(1.76e−2)−	8.4704e−1(2.07e−4)−	8.5171e−1(2.12e−4)−	8.0377e−1(1.66e−3)−	8.5642e−1(7.43e−5)
	30,000	8.4968e−1(1.85e−5)−	8.0455e−1(1.58e−2)−	8.4674e−1(1.75e−4)−	8.5188e−1(1.55e−4)−	7.9524e−1(3.68e−3)−	8.5651e−1(4.73e−5)
TREE3	12,000	8.8948e−1(1.01e−6)+	4.8629e−1(4.41e−3)−	8.0989e−1(1.41e−3)−	8.8948e−1(5.99e−7)+	8.3989e−1(4.76e−3)−	8.7027e−1(3.63e−3)
	30,000	8.9075e−1(4.20e−6)+	4.8720e−1(7.24e−3)−	8.2293e−1(1.29e−3)−	8.9075e−1(8.31e−6)+	8.3731e−1(3.71e−3)−	8.7968e−1(4.78e−3)
TREE4	12,000	9.6643e−1(2.14e−5)+	0.0000e+0(0.00e+0)−	8.0708e−1(2.55e−3)−	6.7868e−1(3.72e−1)≈	7.4912e−1(2.12e−1)≈	8.5404e−1(3.97e−2)
	30,000	9.6834e−1(4.92e−5)+	0.0000e+0(0.00e+0)−	7.9160e−1(2.30e−3)−	9.6848e−1(1.52e−6)+	4.5799e−1(1.57e−1)−	8.5200e−1(5.19e−2)
TREE5	12,000	9.2747e−1(1.00e−5)−	4.2966e−1(3.52e−3)−	9.1625e−1(4.52e−4)−	9.2755e−1(7.88e−5)−	8.3012e−1(3.30e−3)−	9.3588e−1(1.76e−3)
	30,000	9.2698e−1(7.59e−6)−	4.3202e−1(4.71e−3)−	9.1591e−1(6.15e−4)−	9.2709e−1(1.31e−5)−	8.1843e−1(3.94e−3)−	9.3710e−1(9.14e−4)
+/ − / ≈		4/6/0	0/10/0	0/10/0	3/6/1	0/9/1	

+, −, and ≈ indicate that the result of the compared algorithm is significantly better than, worse than, or statistically similar to that of MOZO under the Wilcoxon rank-sum test at the 0.05 significance level, respectively. The best mean result in each row is highlighted in gray.

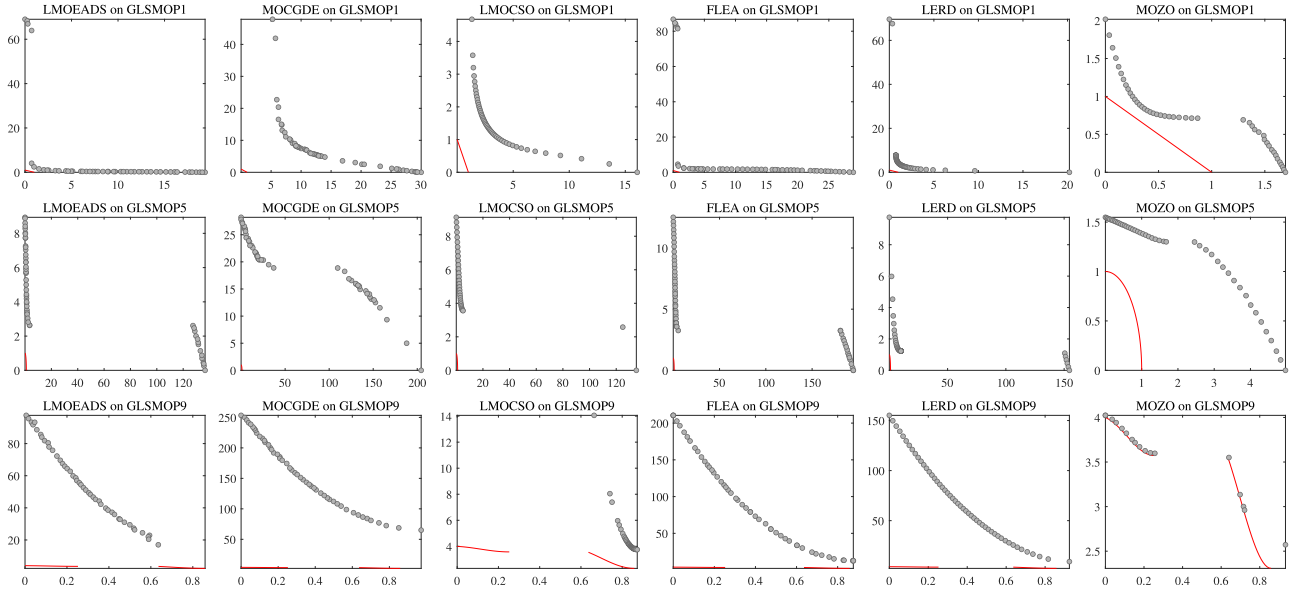


Fig. 8. Non-dominated solutions obtained by LMOEADS, MOCGDE, LMOCSO, FLEA, LERD, and MOZO in the run associated with the median IGD values on bi-objective GLSMOP problems with 10,000 decision variables, where the red lines denote the PF.

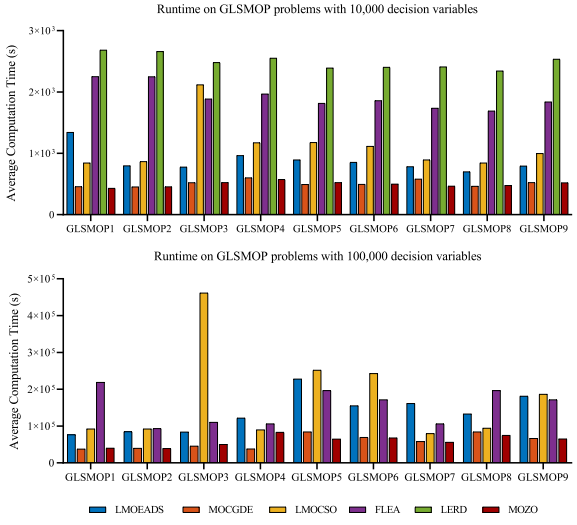


Fig. 9. The runtime consumed by six compared algorithms on nine GLSMOP problems with 10,000 and 100,000 decision variables, respectively.

binary classification [55]. COPA focuses on causal reasoning, requiring the model to choose the more plausible cause or effect for a given premise [56]. BoolQ evaluates open-domain reading comprehension by asking yes/no questions about short passages [57]. RTE examines the model's ability to recognize textual entailment between sentence pairs, which is fundamental to inference tasks [58]. MultiRC is a multi-sentence, multi-answer reading comprehension task that tests the model's ability to perform fine-grained reasoning over long contexts, where each question may have multiple correct answers [59]. These benchmarks provide a comprehensive and challenging testbed for evaluating the generalizability of NLU performance across sentiment analysis, causal inference, factual reasoning, and multi-hop comprehension.

Here, the LLM fine-tuning task is formulated as a bi-objective MOP, i.e.,

$$\min_{\boldsymbol{\theta}} F(\boldsymbol{\theta}) = \left(-\frac{1}{N}\sum_{i=1}^{N}\log P_{\boldsymbol{\theta}}(y_i|i_x), \sum \|\theta\|_2^2\right), \tag{23}$$

where the first objective is the average negative log-likelihood of the predicted targets, while the second objective constrains the ℓ_2 norm (model complexity) of all trainable parameters θ, serving as a regularization to improve generalization. The fundamental training hyperparameters, including batch size, learning rate, optimizer settings,

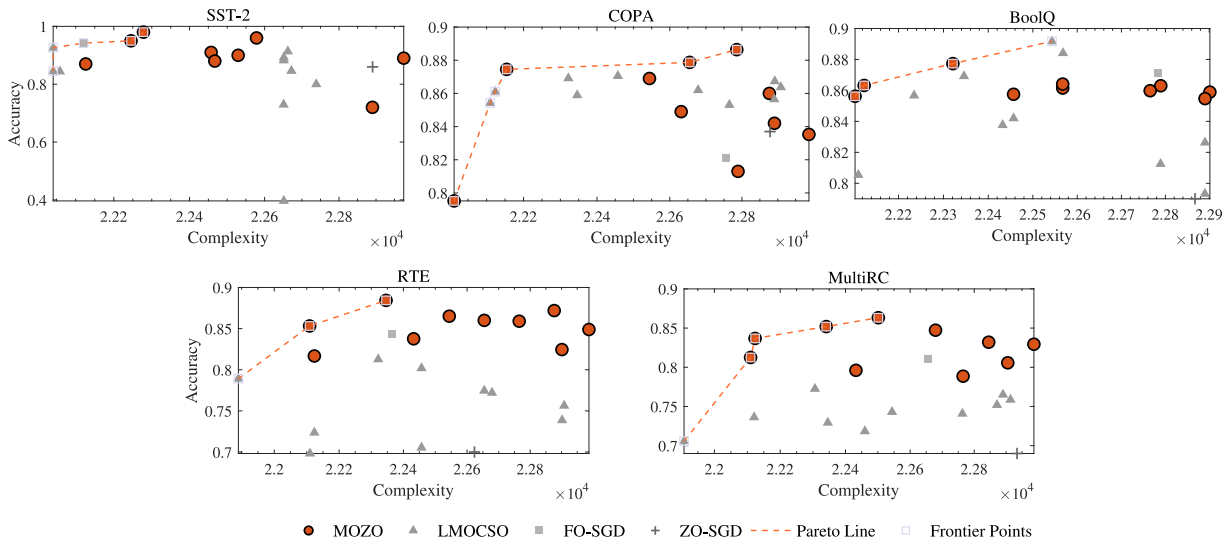


Fig. 10. Comparison of MOZO with compared optimizers on fine-tuning `OPT-6.7B` for the SST-2, COPA, BoolQ, RTE, and MultiRC tasks. Each point represents a trade-off between accuracy and model complexity.

Table 6
Accuracy (%) on five Representative NLU Tasks with Full Fine-tuning of `OPT-2.7B` and `OPT-6.7B`.

`OPT-2.7B`					
Optimizer	SST-2	COPA	BoolQ	RTE	MultiRC
Zero-shot	54.5	74.4	70.9	51.6	47.3
FO-SGD	93.0	**88.9**	**85.3**	82.6	78.7
ZO-SGD	90.7	84.0	77.2	67.2	70.9
LMOCSO	92.5	85.5	82.9	72.3	77.0
MOZO	**95.1**	87.6	84.2	**83.9**	**80.2**
`OPT-6.7B`					
Optimizer	SST-2	COPA	BoolQ	RTE	MultiRC
Zero-shot	58.5	79.0	72.1	59.2	54.9
FO-SGD	94.2	82.1	**87.1**	84.4	81.1
ZO-SGD	86.0	83.7	79.0	70.0	69.0
LMOCSO	89.7	86.2	84.2	77.2	74.3
MOZO	**97.0**	**86.9**	86.3	**86.0**	**83.2**

Zero-shot refers to the performance of the base model without any task-specific fine-tuning.

Table 7
Peak memory cost (in GB), Required GPU Resources, and Runtime Cost (in seconds) of Each Optimizer when Fine-tuning the Full `OPT-6.7B` Model on SST-2.

Optimizer	Memory ⇓	Consumed GPUs ⇓	Runtime cost
FO-SGD	70 GB	3×V100s	32.7 s
LMOCSO	45 GB	2×V100s	21.2 s
MOZO	32 GB	1×V100s	22.4 s
ZO-SGD	32 GB	1×V100s	18.1 s

For large-scale MOEAs that evaluate multiple candidate solutions per generation, the average update time per individual is reported.

and prompt templates, follow the recommended configurations of the ZO-LLM benchmark to ensure fair comparisons.[4]

Table 6 presents the accuracy on five representative NLU tasks under full fine-tuning settings for both `OPT-2.7B` and `OPT-6.7B` LLMs. For MOEAs (MOZO and LMOCSO), the reported results are derived from the median solution in the final population. MOZO delivers competitive or superior performance across both model scales, matching or exceeding FO-SGD in accuracy while maintaining better parameter compactness. In contrast, ZO-SGD lags behind due to degraded gradient estimation in high-dimensional settings. LMOCSO achieves moderate improvements over ZO-SGD but remains inferior to MOZO, underscoring the advantage of problem-specific co-design in MOZO.

Fig. 10 illustrates the trade-off landscapes on SST-2, COPA, BoolQ, RTE, and MultiRC tasks for `OPT-6.7B`, where each point represents a final solution obtained by each optimizer. MOZO consistently produces a well-distributed set of solutions that closely approximate the PF, enabling flexible trade-offs between accuracy and model complexity. In several tasks, MOZO yields solutions that strike favorable balances–achieving competitive accuracy with reduced ℓ_2 complexity, or offering diverse alternatives near the PF. In contrast, FO-SGD often achieves high accuracy without accounting for model complexity, whereas LMOCSO and ZO-SGD often yield suboptimal trade-offs, either underperforming in accuracy or leading to excessive complexity. These results demonstrate that although MOZO may not always produce the absolute best solution in terms of a single metric, it effectively explores the trade-off space and provides deployment-flexible solutions suited for different constraints.

The memory and runtime analysis in Table 7 highlights the practicality of MOZO for full fine-tuning of LLMs. FO-SGD and LMOCSO operate on setups with three and two V100 GPUs, respectively, requiring 70 GB and 45 GB of GPU memory. MOZO completes training with 32 GB of memory on a single V100 GPU, matching ZO-SGD in memory efficiency. In terms of runtime, MOZO incurs a slight overhead compared to ZO-SGD due to the population exchange between the CPU and GPU, but this is effectively offset by its faster convergence. As shown in Fig. 11, MOZO achieves competitive accuracy more quickly and more consistently. Compared with FO-SGD, MOZO achieves comparable convergence speed while incurring significantly lower memory and computational requirements. These results demonstrate that MOZO offers a favorable balance between efficiency and performance, indicating its scalability and cost-effectiveness for resource-constrained scenarios.

Finally, qualitative results from MultiRC, COPA, and RTE (see Table XIV in the Supplementary Materials) indicate that MOZO produces fluent and semantically aligned predictions. These predictions often exhibit correct reasoning and contextual precision. While FO-SGD remains strong, it sometimes fails to capture subtle nuances in expression. ZO-SGD outputs plausible yet ambiguous responses, whereas LMOCSO

[4] https://github.com/ZO-Bench/ZO-LLM.

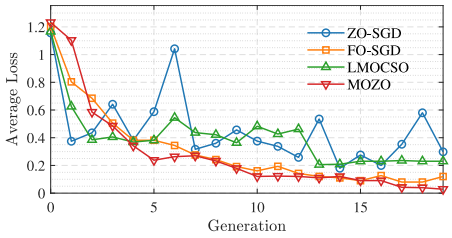


Fig. 11. Convergence profiles of the average training loss on SST-2 when fine-tuning the `OPT-6.7B` model using different optimizers. Each optimizer is executed for the same number of iterations without early stopping.

is more reliable but tends toward under-specified completions. By combining high task accuracy and low resource requirements with consistently robust outputs, MOZO demonstrates practical utility in full fine-tuning scenarios for LLMs.

Overall, these results validate that MOZO approximates state-of-the-art accuracy with fewer resources and also generates high-quality textual predictions. MOZO balances predictive performance, model compactness, and output fidelity, making it a promising option for full LLM fine-tuning, especially in resource-constrained environments.

5. Conclusions and future works

In this work, we propose MOZO, an automated evolutionary zeroth-order optimization method for handling LSMOPs with scaled-up decision variables, integrating ZO optimization with an MOEA paradigm and incorporating reinforcement learning for adaptive configuration. This integration bridges the gap between efficient gradient-free heuristics and the effective global search strategies employed by modern MOEAs, thereby mitigating some scalability challenges associated with the "*Curse of Dimensionality*" rather than eliminating them. The RL-based adaptive mechanism enables state-dependent module configuration during the search process. This reduces the reliance on manual parameter tuning and improves the robustness of MOZO across heterogeneous problem landscapes. Extensive experiments on the GLSMOP and TREE test suites show that MOZO is competitive and often superior to several state-of-the-art MOEAs, especially on high-dimensional bi-objective cases. The mixed results observed on some tri-objective instances also indicate that adaptive selection and objective-conflict handling remain valuable targets for future improvement. Overall, these results highlight the practicality and potential of MOZO in solving real-world LSMOPs, e.g., LLM fine-tuning and large-scale engineering optimization.

Several promising directions remain for future exploration. First, the variance of gradient estimation on certain complex landscapes could be further reduced using advanced variance-reduction techniques or ensemble methods. Second, developing more generalizable RL agents that transfer across different problem classes would enhance its applicability. Finally, extending the adaptive design to handle dynamic optimization problems or incorporating multi-fidelity evaluation strategies could broaden MOZO's scope to even more challenging scenarios.

CRediT authorship contribution statement

Yonglin He: Writing – original draft, Methodology, Investigation. **Cheng He:** Writing – review & editing, Funding acquisition. **Zhichao Lu:** Methodology, Formal analysis, Data curation. **Ye Tian:** Software, Methodology. **Handing Wang:** Validation, Investigation. **Yinglan Feng:** Resources. **Hongbin Li:** Supervision.

Code availability

To support reproducibility, we have prepared a release package containing the implementation code, PPO training settings, random seeds, static module configurations, and LLM fine-tuning scripts. The package is publicly available at the project repository.[5]

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. Supplementary data

Supplementary material related to this article can be found online at https://doi.org/10.1016/j.swevo.2026.102514.

Data availability

No data was used for the research described in the article.

References

[1] A. Gupta, Y.-S. Ong, L. Feng, Insights on transfer optimization: Because experience is the best teacher, TETCI 2 (1) (2017) 51–64.
[2] J.M.P. Cardoso, J.G. de Figueiredo Coutinho, P.C. Diniz, Embedded Computing for High Performance: Efficient Mapping of Computations Using Customization, Code Transformations and Compilation, Morgan Kaufmann, 2017.
[3] S. Sharma, V. Kumar, A comprehensive review on multi-objective optimization techniques: Past, present and future, Arch. Comput. Methods Eng. 29 (2022) 5605–5633.
[4] Z. Wang, Y.-S. Ong, J. Sun, A. Gupta, Q. Zhang, A generator for multiobjective test problems with difficult-to-approximate Pareto front boundaries, IEEE Trans. Evol. Comput. 23 (4) (2018) 556–571.
[5] A. Liefooghe, F. Daolio, S. Verel, B. Derbel, H. Aguirre, K. Tanaka, Landscape-aware performance prediction for evolutionary multiobjective optimization, IEEE Trans. Evol. Comput. 24 (6) (2019) 1063–1077.
[6] W. Li, T. Zhang, R. Wang, S. Huang, J. Liang, Multimodal multi-objective optimization: Comparative study of the state-of-the-art, Swarm Evol. Comput. 77 (2023) 101253.
[7] Z. Hu, K. Shukla, G.E. Karniadakis, K. Kawaguchi, Tackling the curse of dimensionality with physics-informed neural networks, Neural Netw. 176 (2024) 106369.
[8] P. Yang, K. Tang, X. Yao, Turning high-dimensional optimization into computationally expensive optimization, IEEE Trans. Evol. Comput. 22 (1) (2017) 143–156.
[9] Y. Tian, R. Cheng, X. Zhang, Y. Su, Y. Jin, A strengthened dominance relation considering convergence and diversity for evolutionary many-objective optimization, IEEE Trans. Evol. Comput. 23 (2) (2018) 331–345.
[10] J. Liang, X. Ban, K. Yu, B. Qu, K. Qiao, C. Yue, K. Chen, K.C. Tan, A survey on evolutionary constrained multiobjective optimization, IEEE Trans. Evol. Comput. 27 (2) (2022) 201–221.
[11] Y. Tian, L. Si, X. Zhang, R. Cheng, C. He, K.C. Tan, Y. Jin, Evolutionary large-scale multi-objective optimization: A survey, ACM Comput. Surv. 54 (8) (2021) 1–34.
[12] C. He, R. Cheng, C. Zhang, Y. Tian, Q. Chen, X. Yao, Evolutionary large-scale multiobjective optimization for ratio error estimation of voltage transformers, IEEE Trans. Evol. Comput. 24 (5) (2020) 868–881.
[13] H. Tan, R. Cheng, S. Huang, C. He, C. Qiu, F. Yang, P. Luo, RelativeNAS: Relative neural architecture search via slow-fast learning, IEEE Trans. Neural Netw. Learn. 34 (1) (2021) 475–489.
[14] C. He, Y. Tian, Z. Lu, Artificial evolutionary intelligence (AEI): evolutionary computation evolves with large language models, JMC (2024) 1–18.
[15] M.W. Przewozniczek, B. Frej, M. Michal Komarnicki, From direct to directional variable dependencies—Nonsymmetrical dependencies discovery in real-world and theoretical problems, IEEE Trans. Evol. Comput. 29 (2) (2025) 490–504, http://dx.doi.org/10.1109/TEVC.2024.3496193.

[5] https://github.com/ChengHust/ChengHust-Published-Algorithms/tree/main/algorithms/MOZO.

[16] X. Ma, F. Liu, Y. Qi, X. Wang, L. Li, L. Jiao, M. Yin, M. Gong, A multiobjective evolutionary algorithm based on decision variable analyses for multiobjective optimization problems with large-scale variables, IEEE Trans. Evol. Comput. 20 (2) (2015) 275–298.

[17] Y. Tian, C. Lu, X. Zhang, F. Cheng, Y. Jin, A pattern mining-based evolutionary algorithm for large-scale sparse multiobjective optimization problems, IEEE Trans. Cybern. 52 (7) (2020) 6784–6797.

[18] L. Ma, M. Huang, S. Yang, R. Wang, X. Wang, An adaptive localized decision variable analysis approach to large-scale multiobjective and many-objective optimization, IEEE Trans. Cybern. 52 (7) (2021) 6684–6696.

[19] C. He, R. Cheng, L. Li, K.C. Tan, Y. Jin, Large-scale multiobjective optimization via reformulated decision variable analysis, IEEE Trans. Evol. Comput. 28 (1) (2024) 47–61.

[20] W.-J. Hong, P. Yang, K. Tang, Evolutionary computation for large-scale multi-objective optimization: A decade of progresses, IJAC 18 (2) (2021) 155–169.

[21] H. Zille, H. Ishibuchi, S. Mostaghim, Y. Nojima, A framework for large-scale multiobjective optimization based on problem transformation, IEEE Trans. Evol. Comput. 22 (2) (2017) 260–275.

[22] C. He, L. Li, Y. Tian, X. Zhang, R. Cheng, Y. Jin, X. Yao, Accelerating large-scale multiobjective optimization via problem reformulation, IEEE Trans. Evol. Comput. 23 (6) (2019) 949–961.

[23] S. Qin, C. Sun, Y. Jin, Y. Tan, J. Fieldsend, Large-scale evolutionary multiobjective optimization assisted by directed sampling, IEEE Trans. Evol. Comput. 25 (4) (2021) 724–738.

[24] M.N. Omidvar, X. Li, X. Yao, A review of population-based metaheuristics for large-scale black-box global optimization—Part I, IEEE Trans. Evol. Comput. 26 (5) (2021) 802–822.

[25] Y. Tian, X. Zheng, X. Zhang, Y. Jin, Efficient large-scale multiobjective optimization based on a competitive swarm optimizer, IEEE Trans. Cybern. 50 (8) (2019) 3696–3708.

[26] C. He, R. Cheng, D. Yazdani, Adaptive offspring generation for evolutionary large-scale multiobjective optimization, IEEE Trans. Syst. Man Cybern. Syst. 52 (2) (2022) 786–798.

[27] X. Ban, J. Liang, K. Yu, K. Qiao, P.N. Suganthan, Y. Wang, A subspace search-based evolutionary algorithm for large-scale constrained multiobjective optimization and application, IEEE Trans. Cybern. 55 (5) (2025) 2486–2499.

[28] X. Zhang, R. Cheng, Y. Tian, Y. Jin, Evolutionary Large-Scale Multi-Objective Optimization and Applications, John Wiley & Sons, 2024.

[29] L. Li, C. He, R. Cheng, H. Li, L. Pan, Y. Jin, A fast sampling based evolutionary algorithm for million-dimensional multiobjective optimization, Swarm Evol. Comput. 75 (2022) 101181.

[30] Z. Liu, Z. Shen, S. Li, K. Helwegen, D. Huang, K.-T. Cheng, How do Adam and training strategies help BNNs optimization, in: Proc. Int. Conf. Mach. Learn., Vol. 139, PMLR, 2021, pp. 6936–6946.

[31] S.H. Haji, A.M. Abdulazeez, Comparison of optimization techniques based on gradient descent algorithm: A review, PalArch's J. Archaeol. Egypt/Egyptol. 18 (4) (2021) 2715–2743.

[32] Y. Tian, H. Chen, H. Ma, X. Zhang, K.C. Tan, Y. Jin, Integrating conjugate gradients into evolutionary algorithms for large-scale continuous multi-objective optimization, IEEE/CAA J. Autom. Sin. 9 (10) (2022) 1801–1817.

[33] S. Liu, Q. Lin, Y. Tian, K.C. Tan, A variable importance-based differential evolution for large-scale multiobjective optimization, IEEE Trans. Cybern. 52 (12) (2022) 13048–13062.

[34] A.H. Gandomi, X.-S. Yang, S. Talatahari, A.H. Alavi, Metaheuristic algorithms, in: Metaheuristic Applications in Structures and Infrastructures, Elsevier Waltham, 2013, pp. 1–24.

[35] P.-Y. Chen, H. Zhang, Y. Sharma, J. Yi, C.-J. Hsieh, Zoo: Zeroth order optimization based black-box attacks to deep neural networks without training substitute models, in: Proc. ACM Int. Conf. Mach. Learn., 2017, pp. 15–26.

[36] S. Liu, P.-Y. Chen, B. Kailkhura, G. Zhang, A.O. Hero III, P.K. Varshney, A primer on zeroth-order optimization in signal processing and machine learning: Principals, recent advances, and applications, IEEE Signal Process. Mag. 37 (5) (2020) 43–54.

[37] Y. Wang, S. Du, S. Balakrishnan, A. Singh, Stochastic zeroth-order optimization in high dimensions, in: Int. Conf. Artif. Intell. Stat., PMLR, 2018, pp. 1356–1365.

[38] A. Chen, Y. Zhang, J. Jia, J. Diffenderfer, J. Liu, K. Parasyris, Y. Zhang, Z. Zhang, B. Kailkhura, S. Liu, Deepzero: Scaling up zeroth-order optimization for deep model training, 2023, arXiv preprint arXiv:2310.02025.

[39] J. Liu, R. Sarker, S. Elsayed, D. Essam, N. Siswanto, Large-scale evolutionary optimization: A review and comparative study, Swarm Evol. Comput. 85 (2024) 101466.

[40] Y. Zhang, P. Li, J. Hong, J. Li, Y. Zhang, W. Zheng, P.-Y. Chen, J.D. Lee, W. Yin, M. Hong, et al., Revisiting zeroth-order optimization for memory-efficient LLM fine-tuning: A benchmark, 2024, arXiv preprint arXiv:2402.11592.

[41] S. Malladi, T. Gao, E. Nichani, A. Damian, J.D. Lee, D. Chen, S. Arora, Fine-tuning language models with just forward passes, NeuralIPS 36 (2023) 53038–53075.

[42] X. Chen, S. Liu, K. Xu, X. Li, X. Lin, M. Hong, D. Cox, in: H. Wallach, H. Larochelle, A. Beygelzimer, F. d' Alché-Buc, E. Fox, R. Garnett (Eds.), ZO-AdaMM: Zeroth-Order Adaptive Momentum Method for Black-Box Optimization, Vol. 32, Curran Associates, Inc., 2019.

[43] Y. Tian, X. Li, H. Ma, X. Zhang, K.C. Tan, Y. Jin, Deep reinforcement learning based adaptive operator selection for evolutionary multi-objective optimization, TETCI 7 (4) (2022) 1051–1064.

[44] F. Ming, W. Gong, L. Wang, Y. Jin, Constrained multi-objective optimization with deep reinforcement learning assisted operator selection, IEEE/CAA J. Autom. Sin. 11 (4) (2024) 919–931, http://dx.doi.org/10.1109/JAS.2023.123687.

[45] L. Wang, Z. Pan, J. Wang, A review of reinforcement learning based intelligent optimization for manufacturing scheduling, Complex Syst. Model. Simul. 1 (4) (2021) 257–270, http://dx.doi.org/10.23919/CSMS.2021.0027.

[46] S. Li, W. Gong, L. Wang, Q. Gu, Evolutionary multitasking via reinforcement learning, TETCI 8 (1) (2024) 762–775, http://dx.doi.org/10.1109/TETCI.2023.3281876.

[47] Z. Pan, L. Wang, J. Wang, J. Lu, Deep reinforcement learning based optimization algorithm for permutation flow-shop scheduling, TETCI 7 (4) (2023) 983–994, http://dx.doi.org/10.1109/TETCI.2021.3098354.

[48] M. Nazari, A. Oroojlooy, L. Snyder, M. Takác, Reinforcement learning for solving the vehicle routing problem, NeuralIPS 31 (2018).

[49] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, 2017, arXiv preprint arXiv:1707.06347.

[50] H. Gu, H. Wang, J. Ma, Test suites and performance of algorithms in large-scale multiobjective evolutionary optimization, in: IEEE Congr. Evol. Comput., IEEE, 2024, pp. 1–8.

[51] C.A.C. Coello, N.C. Cortés, Solving multiobjective optimization problems using an artificial immune system, GPEM 6 (2) (2005) 163–190.

[52] K. Shang, H. Ishibuchi, L. He, L.M. Pang, A survey on the hypervolume indicator in evolutionary multiobjective optimization, IEEE Trans. Evol. Comput. 25 (1) (2020) 1–20.

[53] Y. Yan, Q. Liu, Y. Li, Paradox-free analysis for comparing the performance of optimization algorithms, IEEE Trans. Evol. Comput. 27 (5) (2022) 1275–1287.

[54] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Chen, C. Dewan, M. Diab, X. Li, X.V. Lin, et al., Opt: Open pre-trained transformer language models, 2022, arXiv preprint arXiv:2205.01068.

[55] R. Socher, A. Perelygin, J. Wu, J. Chuang, C.D. Manning, A.Y. Ng, C. Potts, Recursive deep models for semantic compositionality over a sentiment treebank, in: Conf. Empir. Methods Nat. Lang. Process., 2013, pp. 1631–1642.

[56] M. Roemmele, C.A. Bejan, A.S. Gordon, Choice of plausible alternatives: An evaluation of commonsense causal reasoning, in: AAAI Spring Symp., Tech. Rep., 2011, pp. 90–95.

[57] C. Clark, K. Lee, M.-W. Chang, T. Kwiatkowski, M. Collins, K. Toutanova, BoolQ: Exploring the surprising difficulty of natural yes/no questions, 2019, arXiv preprint arXiv:1905.10044.

[58] I. Dagan, O. Glickman, B. Magnini, The PASCAL recognising textual entailment challenge, in: Machine Learning Challenges: Evaluating Predictive Uncertainty, Visual Object Classification, and Recognising Textual Entailment, Southampton, United Kingdom, 2006, pp. 177–190.

[59] D. Khashabi, S. Chaturvedi, M. Roth, S. Upadhyay, D. Roth, Looking beyond the surface: A challenge set for reading comprehension over multiple sentences, in: NAACL HLT - Conf. N. Am. Chapter Assoc. Comput. Linguistics: Hum. Lang. Technol.- Proc. Conf., 2018, pp. 252–262.